

Documentació Tècnica

Practica1_Serial_USB



Juan Vidal Moreno
Ruben Sàenzo Tarancón
SMX2B
2025-2026

Índex

1. Portada
2. Índex
3. Títol i objectiu de la pràctica
4. Explicació de conceptes clau
 - 4.1. Classe, objecte i mètode
 - 4.2. Relació amb l'objecte Serial
5. Esquema de connexions
6. Diagrama de flux del programa
7. Flux de dades i procés de lectura/escriptura
 - 7.1. Diagrama visual del flux de dades
 - 7.2. Explicació de les funcions principals
 - 7.3. Codi final de la pràctica
8. Proves i vídeo demostratiu
9. Seguiment del projecte a GitHub

3. Títol i objectiu de la pràctica

Objectiu

L'objectiu principal d'aquesta pràctica és aprendre a establir una comunicació bidireccional entre una placa ESP32 i un ordinador mitjançant el port sèrie USB. A més, s'introdueixen els conceptes bàsics de la programació orientada a objectes en C++: classes, objectes i mètodes.

Descripció de la pràctica

Durant la pràctica he programat una ESP32 perquè pugui rebre ordres de text des del Monitor Sèrie de l'Arduino IDE. Segons l'ordre enviada (ON o OFF), la placa encén o apaga un LED connectat a un dels seus pins digitals. Si l'ordre no es reconeix, el programa mostra un missatge d'error.

Tot el procés m'ha permès entendre com funciona la lectura i escriptura de dades a través del port USB, i com aplicar la lògica de control dins d'un bucle continu.

4. Explicació de conceptes clau

4.1 – Què és una classe?

Una classe és com una plantilla que defineix l'estructura d'un objecte: quines dades emmagatzemarà i quines accions podrà fer. Per exemple, si imaginem una classe **Cotxe**, podem dir que tindrà una variable **velocitat** i mètodes com **encendre()** o **apagar()**. La classe en si no és cap cotxe concret, sinó la definició d'allò que serà qualsevol cotxe que creem a partir d'ella.

```
class Cotxe {  
public:  
    int velocitat;  
    void encendre() { Serial.println("Motor encens"); }  
    void apagar() { Serial.println("Motor apagat"); }  
};
```

4.2 – Què és un objecte?

Un objecte és una instància concreta d'una classe. Si la classe és la plantilla, l'objecte és el resultat de construir-la. Podem tenir tants objectes com vulguem a partir d'una mateixa classe, i cadascun és independent dels altres.

```
Cotxe meuCotxe; // Creem un objecte de la classe Cotxe
meuCotxe.velocitat = 50;
meuCotxe.encendre();
```

4.3 – Què és un mètode?

Un mètode és una funció que pertany a una classe. Es pot definir dins de la classe directament, o declarar-se dins i definir-se fora usant l'operador ::. Els mètodes permeten que els objectes facin accions o modifiquin el seu estat intern.

```
// Definició fora de la classe
void Cotxe::encendre() {
    Serial.println("Motor ences");
}
```

4.4 – Relació amb l'objecte Serial

A Arduino, **Serial** és un exemple perfecte d'objecte: és una instància ja creada de la classe **HardwareSerial**. No cal que el creem nosaltres; l'entorn d'Arduino ja el posa a disposició del programa. La idea és la mateixa que amb **meuCotxe.encendre()**: fem servir el nom de l'objecte seguit d'un punt i el nom del mètode.

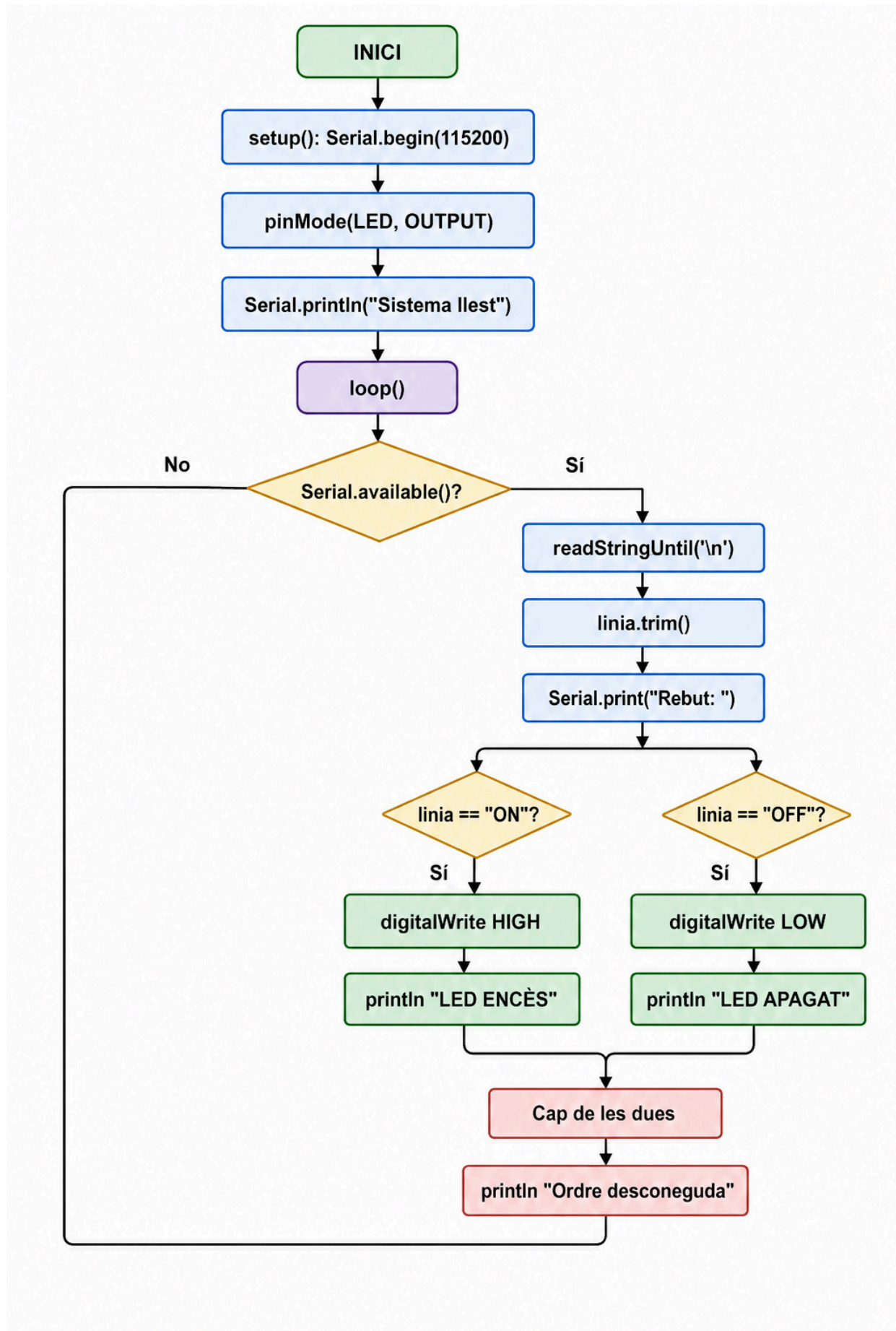
Mètode	Funció
Serial.begin(115200)	Inicialitza el port USB a 115200 baud
Serial.available()	Comprova si hi ha dades pendents de llegir
Serial.read()	Llegeix un caràcter del port sèrie
Serial.readStringUntil('\n')	Llegeix una línia sencera fins al salt de línia

Serial.print() / println()	Envia text cap al Monitor Sèrie
----------------------------	---------------------------------

5. Esquema de connexions

Component	Connexió
ESP32 – Pin 2	Ànode del LED (+) a través de resistència 220Ω
ESP32 – GND	Càtode del LED (pota curta)
Cable USB	Connexió ESP32 ↔ ordinador (alimentació + comunicació sèrie)

6. Diagrama de flux del programa



7.2 – Explicació de les funcions principals

Serial.available()

Retorna el nombre de bytes disponibles al buffer del port sèrie. Si és més gran que zero, hi ha dades esperant. Ho fem servir al loop() per no bloquejar el programa quan no arriba res.

Serial.readStringUntil('\n')

Llegeix tots els caràcters fins que troba un salt de línia (\n). Quan l'usuari prem Enter al Monitor Sèrie, s'envia el \n i la funció retorna el text complet.

linia.trim()

Elimina espais en blanc al principi i al final de la cadena, i el caràcter \r que Windows afegeix als salts de línia. Sense aquesta funció, comparar "ON" amb "ON\r" donaria fals.

linia.equalsIgnoreCase("ON")

Compara la cadena ignorant majúscules/minúscules. D'aquesta manera, "on", "On" i "ON" s'accepten com a vàlids. És millor que == perquè l'usuari no ha de ser precís.

Serial.println()

Envia text de tornada cap al Monitor Sèrie, afegint automàticament un salt de línia al final. print() fa el mateix però sense el salt. Fem servir println() per mostrar les respostes.

7.3 – Codi final de la pràctica

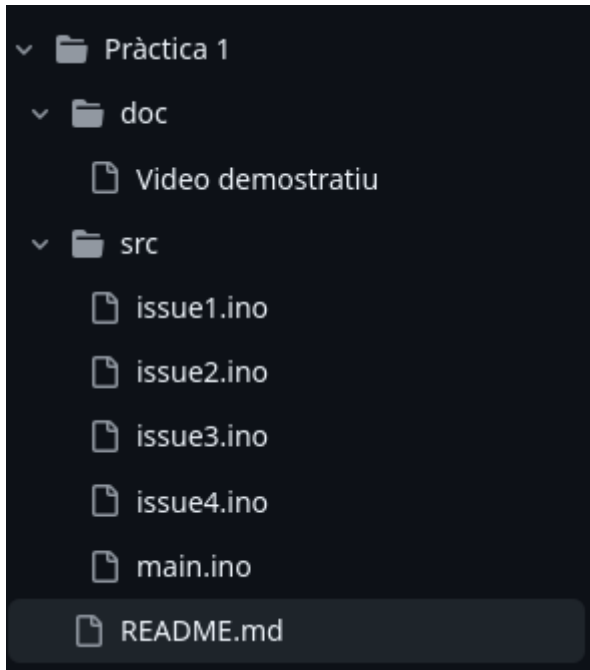
```
Code Blame 53 lines (44 loc) - 1.2 KB

1  const int tempPin = 2;
2  const int ldrPin = 4;
3  const int ledPin = 25;
4
5  float temperatura = 0.0;
6  int valorTemp = 0;
7  int valorLDR = 0;
8
9  void setup() {
10     pinMode(ledPin, OUTPUT);
11     Serial.begin(115200);
12 }
13
14 void loop() {
15     valorTemp = analogRead(tempPin);
16     float voltage = valorTemp * (3.3 / 4095.0);
17     temperatura = voltage * 100.0;
18
19     valorLDR = analogRead(ldrPin);
20
21     if (temperatura > 15.0) {
22         digitalWrite(ledPin, HIGH);
23         Serial.println("ALERTA: Sobreescalfament CPD!");
24     } else {
25         digitalWrite(ledPin, LOW);
26     }
27
28     if (valorLDR > 1000) {
29         Serial.println("AVÍS: Porta oberta o llum encesa");
30     }
31
32     if (Serial.available()) {
33         String linea = Serial.readStringUntil('\n');
34         linea.trim();
35         linea.toLowerCase();
36         if (linea == "status"){
37             Serial.print("Temperatura: ");
38             Serial.print(temperatura);
39             Serial.println(" °C");
40             Serial.print("Il·luminositat: ");
41             Serial.println(valorLDR);
42         } else if (linea == "off" and digitalRead(25) == HIGH) {
43             digitalWrite(ledPin, LOW);
44             Serial.println("Led apagat manualment");
45         } else if (linea == "off" and digitalRead(25) == LOW){
46
47         } else {
48             Serial.println("Error: ordre desconeguda.");
49         }
50     }
51     delay(2000);
52
53 }
```


9. Seguiment del projecte a GitHub

9.1 – Creació del repositori

Hem creat un repositori públic a GitHub amb el nom Practica 1. L'estructura de carpetes és la que es demana a l'enunciat:



9.4 – Flux de treball i commits

Hem anat movent cada issue de **To do** a **Doing** quan el començava, i a **Done** en acabar-lo. Cada commit al repositori fa referència a l'issue que tanca:

The screenshot shows a Kanban board titled "Pràctica 1 - Serial USB". At the top, there is a "View 1" button and a search bar labeled "Filter by keyword or by field". The board is divided into three columns:

- Todo (0):** This item hasn't been started.
- In Progress (0):** This is actively being worked on.
- Done (6):** This has been completed. It contains six items, each with a checkmark icon and a green checkmark icon:

Issue ID	Task Description
Projecte-Dani-Soldevila-2na-Convocatoria #3	Llegir línia des del Monitor Sèrie
Projecte-Dani-Soldevila-2na-Convocatoria #2	Inicialitzar port sèrie USB
Projecte-Dani-Soldevila-2na-Convocatoria #4	Mostrar dades rebudes
Projecte-Dani-Soldevila-2na-Convocatoria #5	Controlar LED amb ordres textuais
Projecte-Dani-Soldevila-2na-Convocatoria #6	Documentació tècnica
Projecte-Dani-Soldevila-2na-Convocatoria #7	Vídeo demostració